

The HyMeKo System-Engineering View

A manifesto — one source, many views

Companion to *HyMeKo Capability Evidence Ledger*. Compiled 2026-06-20.

1. Thesis

One source, many views. A robotic system is described *once*, as a signed hypergraph in HyMeKo. Every artifact a downstream tool needs — URDF, SDF, MJCF, a Gazebo launch bundle, a SysML block diagram, a Lean4 proof obligation, a DOT graph, the figure in a paper, an RL *AgentSpec* — is a **projection** (a view) of that single source, never a parallel hand-maintained copy.

This is **Model-Based Systems Engineering (MBSE)** taken literally. In INCOSE’s terms, MBSE is *the formalized application of modelling to support system requirements, design, analysis, verification and validation, beginning in the conceptual design phase and continuing throughout development and later life-cycle phases* (INCOSE SE Vision; the direction of travel of the discipline through SE Vision 2035). HyMeKo is the **authoritative source of truth** that vision calls for: the model is not documentation *about* the system; the model **is** the system of record. Code, simulation assets, and diagrams are generated downstream and are, by construction, consistent with each other because they share one upstream cause — a single point on the **digital thread**.

The failure mode this exists to kill is the *document-centric* trap: the same robot encoded three times — once in a URDF, once in a slide, once in a training script — drifting out of sync the moment anyone edits one of them. Under MBSE there is nothing to keep in sync, because there is only one editable artifact and everything else is derived from it.

2. Two layers, separated on purpose

Every HyMeKo description separates **vocabulary** from **instance**.

Layer	What it is	Example
Vocabulary (<code>meta_*</code>)	The <i>kinds</i> — link, joint, axis, sensor, context, signal, aggregation. Declared once per ecosystem.	<code>meta_kinematics.hymeko</code> , <code>meta_context.hymeko</code>
Instance	The <i>specific</i> robot — its links, joints, context graph. Imports a vocabulary; declares only what is unique.	<code>wam.hymeko</code> , <code>hymeko_robot_reuse.hymeko</code>

A vocabulary is amortised across **every** system that imports it. The cost of defining “what a revolute joint is” is paid once, not once per robot.

3. Why this is the selling point, not a refactor

Measured (`reports/2026-06-20-mdsd-reuse-scenario.md`). The ROS2 demo robot, authored both ways — bare inline types (kept as a frozen control) vs. imported vocabulary:

- **Length:** 189 → 126 lines (−33%); 108 → 76 code lines for *one* robot. The shared vocabulary is paid once, so the marginal saving for the *N*-th robot is the full re-declaration.

- **Semantics, not just brevity:** the imported form carries *typed* joints with real axis vectors and limits; URDF/SDF emit valid <joint> elements. The bare form’s generic `joint` emits **zero**. Reuse is the difference between a description that *projects to a working robot* and one that does not.

The length number is the headline; the semantic number is the argument.

4. What counts as a “view” (ISO/IEC/IEEE 42010)

View is used in the precise sense of **ISO/IEC/IEEE 42010**, the architecture-description standard INCOSE builds on: a **view** addresses a set of **stakeholder concerns** from a **viewpoint**. The architecture description *is* the signed hypergraph; each emitter is a viewpoint; each emitted artifact is a view.

Stakeholder concern	Viewpoint (emitter)	View (artifact)
“Will it move correctly in sim?”	URDF / SDF / MJCF / Gazebo	simulation scene
“How does it decompose into blocks & ports?”	SysML	block / IBD diagram
“Is this property provable?”	Lean4	proof obligation
“What is the structure?”	DOT / Mermaid / WASM editor	graph
“What is the learning problem?”	RL AgentSpec	obs + action + reward + vertices
“Did the projection lose information?”	HyMeKo → HyMeKo	canonical round-trip

The views are **not** independent documents to be reconciled — they are consistent by construction. Adding a view means writing one viewpoint against the IR, never asking the author to maintain another copy.

5. The boundary rules (so the principle does not rot)

1. **Vocabulary is imported, never re-declared.** Inline kinds where a `meta_*` profile already declares them is duplication in the costume of self-containment. The only exception is a **frozen reference artifact**, marked as such.
2. **Parametric difference → configuration; structural difference → a different description.** Different link lengths = same description, new field values. Different topology = a different instance graph. Never a runtime `if`.
3. **A view is read-only downstream.** Never hand-edit a generated URDF. If a view is wrong, the bug is in the source or the emitter.
4. **One source per system, even across tools.** The robot that drives the MuJoCo env, the Gazebo demo, and the SysML diagram is *one* `.hymeko`.

6. Alignment with INCOSE systems-engineering practice

This manifesto is the project’s reading of established SE doctrine, applied to a hypergraph core.

HyMeKo	Systems-engineering concept	Source
The .hymeko source	Authoritative source of truth / single system model	INCOSE MBSE, SE Vision 2035
Emitter + emitted artifact	Viewpoint + view addressing a concern	ISO/IEC/IEEE 42010
Vocabulary vs instance	Separation of concerns; reference vs system data	ISO/IEC/IEEE 15288
Cross-file import + canonical hash	Traceability; configuration identity	INCOSE Handbook
.hymeko → URDF/SysML/Lean4/...	The digital thread across life-cycle activities	Digital Engineering
Round-trip + validation-gated emit	Verification & Validation	ISO/IEC/IEEE 15288

Where this sits on the Vee. The SE *Vee* has a left leg (decomposition & definition: needs → requirements → architecture → design) and a right leg (integration & verification). HyMeKo’s authoring sits on the **left leg**, captured once as the hypergraph; the emitters carry that single definition rightward into each V&V activity — the same model that *defines* the robot is the one *verified* against each consumer.

Verification vs validation, kept distinct (INCOSE’s two questions).

- *Verification* — “*did we build the system right?*” Are the views **faithful** to the source? Mechanically checkable: the HyMeKo→HyMeKo round-trip and the canonical hash are verification gates. The MDSD reuse result is a verification statement — the imported description projects to the same kinematic structure, hash-confirmed.
- *Validation* — “*did we build the right system?*” Does the model meet the need? This the framework **cannot** assert for you; it requires running the robot, the sim, the experiment. The capability ledger is deliberately honest about this line: PROVEN rows are largely verification; SHIPPED / RESEARCH rows are where validation is still open.

Why a hypergraph, in SE terms. SysML models systems as blocks with ports and pairwise connections. Many real engineering relations are **not** pairwise — a constraint over three quantities, a context fusing five signals, a signed balance over a cycle. Forcing those into binary connectors loses information at authoring time. The signed hypergraph (GGK 4-tuple $K = (B, G, \mu, r)$) keeps higher-order, signed, cross-context relations as first-class, and the pairwise SysML view is recovered by projection — never the other way around.

7. The standing invariant

*If a fact about a system is true, it is stated **once** in HyMeKo and **derived everywhere else**. Any place a fact is restated by hand is a latent inconsistency, and is treated as a defect — not a convenience.*

This is the single-authoritative-source principle of MBSE stated as an operational rule, and it is what makes traceability mechanical rather than aspirational: every view traces to exactly one source element, and the canonical hash detects when that link is broken.